Sérgio de Paiva Ximenes Filho

Análise empírica do Insertion Sort e do Merge Sort: tempos de execução, estatística descritiva e regressão polinomial

Fortaleza

2026

Sérgio de Paiva Ximenes Filho

Análise empírica do Insertion Sort e do Merge Sort: tempos de execução, estatística descritiva e regressão polinomial

Relatório da atividade prática de implementação (AV2) apresentado à disciplina T198 – Construção e Análise de Algoritmos, do curso de Bacharelado em Ciência da Computação da Universidade de Fortaleza.

Universidade de Fortaleza – UNIFOR

Centro de Ciências Tecnológicas

Bacharelado em Ciência da Computação

Orientador: Prof. Bruno Lopes

Fortaleza

2026

RESUMO

Este relatório apresenta uma análise empírica do tempo de execução dos algoritmos de ordenação Insertion Sort e Merge Sort, implementados na linguagem C, com o objetivo de verificar se o comportamento observado na prática confirma as ordens de crescimento previstas pela análise assintótica. Os dois algoritmos foram executados sobre vetores de 1.000 a 1.000.000 elementos em três cenários de entrada (crescente, aleatório e decrescente), com 50 repetições por combinação e medição de tempo pela função `clock_gettime()` com o relógio `CLOCK_MONOTONIC`. Para cada combinação foram calculados média, desvio padrão, intervalo de confiança de 95 %, mínimo e máximo, e foram ajustados polinômios de grau 1 a 3 às médias pelo método dos mínimos quadrados. Os resultados confirmaram o comportamento quadrático do Insertion Sort nos casos médio e pior (expoentes empíricos de 1,96 e 1,99, razão próxima de 4 ao dobrar n e coeficiente de determinação de 0,999999 para o polinômio de grau 2), o comportamento linear do Insertion Sort no melhor caso e o crescimento quase linear do Merge Sort, compatível com $\Theta(n \log n)$. O Merge Sort foi significativamente mais rápido que o Insertion Sort em todos os tamanhos testados nos casos médio e pior, chegando a ser cerca de 632 vezes mais rápido com $n = 1.000.000$ no caso médio, enquanto o Insertion Sort foi o mais rápido em todos os tamanhos do melhor caso. Conclui-se que a análise empírica confirmou as previsões teóricas e evidenciou aspectos que a notação assintótica omite, como as constantes multiplicativas e a sensibilidade de cada algoritmo à configuração da entrada.

Palavras-chave: algoritmos de ordenação; insertion sort; merge sort; análise empírica; regressão polinomial.

LISTA DE ILUSTRAÇÕES

Quadro 1 – Especificações do ambiente de execução 7
Figura 1 – Tempo médio de execução no melhor caso (vetor crescente) 15
Figura 2 – Tempo médio de execução no caso médio (vetor aleatório) 15
Figura 3 – Tempo médio de execução no pior caso (vetor decrescente) 16
Figura 4 – Tempo médio em função de n em escala log-log 18
Figura 5 – Médias observadas e curvas ajustadas no melhor caso 25
Figura 6 – Médias observadas e curvas ajustadas no caso médio 25
Figura 7 – Médias observadas e curvas ajustadas no pior caso 26

LISTA DE TABELAS

Tabela 1 – Insertion Sort, melhor caso: estatísticas do tempo de execução (ms) . . 12
Tabela 2 – Insertion Sort, caso médio: estatísticas do tempo de execução (ms) . . 12
Tabela 3 – Insertion Sort, pior caso: estatísticas do tempo de execução (ms) 13
Tabela 4 – Merge Sort, melhor caso: estatísticas do tempo de execução (ms) . . . 13
Tabela 5 – Merge Sort, caso médio: estatísticas do tempo de execução (ms) 14
Tabela 6 – Merge Sort, pior caso: estatísticas do tempo de execução (ms) 14
Tabela 7 – Expoentes empíricos k, obtidos pela inclinação no gráfico log-log 18
Tabela 8 – Razão $T(2n)/T(n)$ entre os tempos médios ao dobrar n 19
Tabela 9 – Comparação entre os algoritmos no melhor caso 20
Tabela 10 – Comparação entre os algoritmos no caso médio 20
Tabela 11 – Comparação entre os algoritmos no pior caso 21
Tabela 12 – Funções obtidas pela regressão polinomial para o Insertion Sort 24
Tabela 13 – Funções obtidas pela regressão polinomial para o Merge Sort 24

SUMÁRIO

1 INTRODUÇÃO 6

2 METODOLOGIA 7
2.1 Ambiente de execução 7
2.2 Implementação dos algoritmos 8
2.3 Medição do tempo 8
2.4 Geração das entradas 9
2.5 Coleta e organização dos dados 9
2.6 Estatística descritiva 10
2.7 Regressão polinomial e indicadores de crescimento 10

3 RESULTADOS 12
3.1 Estatísticas do Insertion Sort 12
3.2 Estatísticas do Merge Sort 13
3.3 Gráficos comparativos 14

4 ANÁLISE 17
4.1 Ordens de crescimento observadas 17
4.2 Comportamento ao dobrar o tamanho da entrada 19
4.3 Comparação entre os algoritmos 20
4.4 Diferenças entre os cenários 22
4.5 Variabilidade e intervalos de confiança 22
4.6 Regressão polinomial 23
4.6.1 Insertion Sort 26
4.6.2 Merge Sort 27

5 CONCLUSÃO 29

REFERÊNCIAS 30

APÊNDICES 31

APÊNDICE A – REPRODUÇÃO DO EXPERIMENTO 32

1 INTRODUÇÃO

A análise assintótica descreve como o custo de um algoritmo cresce em função do tamanho da entrada, abstraindo constantes multiplicativas e termos de ordem inferior (Cormen *et al.*, 2012). Essa abstração permite comparar algoritmos independentemente de máquina, mas não informa quanto tempo um programa leva de fato para executar, nem a partir de que tamanho de entrada um algoritmo assintoticamente melhor passa a compensar na prática. Fatores como o compilador e suas otimizações, a hierarquia de memória, o sistema operacional e a própria configuração da entrada afetam o tempo real de execução. Por isso, a análise experimental é um complemento necessário à análise teórica: ela permite verificar se as previsões se confirmam e quantificar os aspectos que a notação assintótica deixa de fora (Johnson, 2002; McGeoch, 2012).

Os algoritmos de ordenação são um caso adequado para esse tipo de estudo, porque possuem análise teórica bem estabelecida e comportamento que depende fortemente da entrada. O Insertion Sort executa $\Theta(n)$ operações quando o vetor já está ordenado, mas $\Theta(n^2)$ no caso médio e no pior caso, quando o vetor está em ordem decrescente. Já o Merge Sort, baseado na estratégia de divisão e conquista, executa $\Theta(n \log n)$ operações em qualquer caso, ao custo de memória auxiliar proporcional a n (Cormen *et al.*, 2012).

O objetivo deste trabalho foi implementar os dois algoritmos na linguagem C, medir seus tempos de execução para dez tamanhos de entrada, entre 1.000 e 1.000.000 elementos, em três cenários (melhor caso, caso médio e pior caso), e analisar os dados por meio de estatística descritiva e regressão polinomial. Com base na teoria, esperava-se observar que:

a) o tempo do Insertion Sort cresce de forma aproximadamente quadrática nos casos médio e pior, multiplicando-se por cerca de 4 a cada vez que n dobra, e de forma linear no melhor caso;

b) o tempo do Merge Sort cresce de forma ligeiramente superior à linear em todos os cenários, multiplicando-se por pouco mais de 2 quando n dobra;

c) o Merge Sort supera o Insertion Sort nos casos médio e pior a partir de algum tamanho de entrada, com a diferença aumentando rapidamente com n, enquanto no melhor caso o Insertion Sort permanece mais rápido;

d) o polinômio de grau 2 se ajusta bem aos dados do Insertion Sort nos casos médio e pior.

O Capítulo 2 descreve o ambiente e o procedimento experimental. O Capítulo 3 apresenta os dados coletados, sem interpretação. O Capítulo 4 interpreta os resultados à luz da teoria, incluindo a regressão polinomial, e o Capítulo 5 sintetiza as conclusões.

2 METODOLOGIA

Este capítulo descreve o ambiente de execução, a implementação, a geração das entradas, a coleta dos dados e os métodos estatísticos empregados, com o detalhamento necessário para que o experimento possa ser reproduzido. O Apêndice A lista os comandos utilizados.

2.1 Ambiente de execução

Os experimentos foram executados em um computador com as especificações do Quadro 1.

Quadro 1 – Especificações do ambiente de execução

Item	**Especificação**
Processador	Intel Core i5-12400 (12.ª geração), frequência base de 2,5 GHz, 6 núcleos e 12 *threads*, cache L2 de 7,5 MiB e L3 de 18 MiB
Memória RAM	32 GB instalados (31,8 GB utilizáveis pelo Windows), dos quais 15,5 GiB disponíveis para a máquina virtual do WSL 2
Sistema hospedeiro	Microsoft Windows 11 (versão 10.0.26200)
Sistema de execução	Ubuntu 26.04.1 LTS no Windows Subsystem for Linux 2 (WSL 2), núcleo Linux 6.18.33.2-microsoft-standard-WSL2
Compilador	GCC 15.2.0
Flags de compilação	`-Wall -Wextra -O2`
Análise de dados	Python 3.14.4 com NumPy 2.5.3, pandas 3.0.6, SciPy 1.18.1 e Matplotlib 3.11.2

Fonte: Elaborado pelo autor (2026).

O programa foi compilado pelo `Makefile` do projeto com o comando `gcc -Wall -Wextra -O2 main.c sorts.c -o main`. As opções `-Wall` e `-Wextra` apenas habilitam avisos do compilador e não alteram o código gerado. A opção `-O2` habilita o conjunto de otimizações do GCC que não envolvem uma troca entre espaço e velocidade (Free Software Foundation, 2026). Ela foi escolhida porque representa a forma como programas em C costumam ser compilados para uso real. As otimizações reduzem as constantes multiplicativas do tempo de execução, mas não alteram o número de comparações e movimentações que cada algoritmo realiza; portanto, não alteram a ordem de crescimento. Os tempos absolutos seriam outros com `-O0` ou `-O3`, mas as razões entre tamanhos de entrada e os expoentes analisados no Capítulo 4 devem se manter.

Durante as execuções, o computador foi utilizado normalmente, sem isolamento de núcleos de processamento nem desativação do ajuste dinâmico de frequência do processador. Esses fatores, somados à camada de virtualização do WSL 2, são fontes de variação entre as repetições e foram considerados na análise da variabilidade (Seção 4.5).

2.2 Implementação dos algoritmos

Os algoritmos foram implementados no arquivo `sorts.c`, com base nos pseudocódigos de Cormen *et al.* (2012), ambos ordenando vetores de inteiros (`int`) em ordem crescente:

a) **Insertion Sort** (`insertion_sort`): percorre o vetor da segunda posição em diante; cada elemento é guardado em uma variável auxiliar (`chave`) e os elementos maiores à sua esquerda são deslocados uma posição para a direita até que a posição correta seja encontrada. A ordenação é feita no próprio vetor, sem memória adicional;

b) **Merge Sort** (`merge_sort`): versão recursiva descendente, que divide o intervalo $[p, r]$ ao meio, ordena cada metade recursivamente e intercala as duas metades com a função `merge`. Cada chamada de `merge` aloca com `malloc` dois vetores auxiliares, com as cópias das duas metades, e os libera com `free` ao final; o custo dessas alocações faz parte do tempo medido. A função `merge_sort_wrapper(v, n)` adapta a assinatura para `(int v[], int n)`, igual à do Insertion Sort, o que permite medir os dois algoritmos com a mesma função.

2.3 Medição do tempo

O tempo foi medido pela função `medir`, do arquivo `main.c`, que recebe um ponteiro para a função de ordenação, chama `clock_gettime(CLOCK_MONOTONIC, ...)` imediatamente antes e imediatamente depois da ordenação e devolve a diferença em segundos, com resolução de nanossegundos. Apenas a chamada da ordenação fica entre as duas leituras do relógio; a geração do vetor e a gravação em arquivo ficam fora do intervalo medido.

O relógio `CLOCK_MONOTONIC` foi escolhido porque mede o tempo decorrido a partir de um ponto fixo e não sofre os saltos que afetam o relógio de tempo real do sistema (`CLOCK_REALTIME`), como ajustes manuais da hora ou sincronizações automáticas (Linux man-pages project, 2026). Ele também oferece resolução muito superior à da função `clock()` da biblioteca padrão, que mede o tempo de processador em unidades de `CLOCKS_PER_SEC`. Por medir o tempo decorrido, e não apenas o tempo de processador, o valor obtido inclui eventuais interrupções do processo pelo sistema operacional, o que contribui para a variabilidade observada.

2.4 Geração das entradas

Para cada cenário, o vetor foi preenchido pela função `preencher`, que seleciona uma das funções a seguir:

a) **melhor caso** (`preencher_crescente`): valores de 1 a n em ordem crescente (`v[k] = k + 1`). Para o Insertion Sort é o melhor caso, pois nenhum elemento precisa ser deslocado;

b) **caso médio** (`preencher_aleatorio`): valores gerados pela função `rand()` da biblioteca padrão, no intervalo de 0 a `RAND_MAX` ($2^{31} - 1$ na biblioteca GNU C). Antes de cada preenchimento, o gerador é inicializado com `srand(j + 1)`, em que j é o índice da repetição, de 0 a 49. Assim, cada uma das 50 repetições usa um vetor aleatório diferente, como exige o enunciado;

c) **pior caso** (`preencher_decrescente`): valores de n a 1 em ordem decrescente (`v[k] = n - k`). Para o Insertion Sort é o pior caso, pois cada elemento precisa ser deslocado até o início do vetor.

Para garantir que os dois algoritmos ordenassem exatamente o mesmo vetor em cada teste, o vetor foi gerado novamente, de forma determinística, imediatamente antes de cada execução, e não apenas uma vez. Nos cenários crescente e decrescente o conteúdo depende somente de n. No caso médio, como o gerador é reinicializado com a mesma semente (`srand(j + 1)`) antes da repetição j de cada algoritmo, a sequência de números produzida por `rand()` é idêntica. Dessa forma, a repetição j do Insertion Sort e a repetição j do Merge Sort recebem vetores com conteúdo idêntico, o que tem o mesmo efeito de gerar o vetor uma vez e copiá-lo para cada algoritmo. A regeneração também é necessária porque a ordenação é feita no próprio vetor: sem ela, o segundo algoritmo receberia um vetor já ordenado.

2.5 Coleta e organização dos dados

Para cada combinação de algoritmo, cenário e tamanho $n \in \{1.000, 2.000, 5.000, 10.000, 20.000, 50.000, 100.000, 200.000, 500.000, 1.000.000\}$, foram realizadas 50 repetições consecutivas. O laço principal percorre os cenários, depois os tamanhos e, para cada tamanho, executa as 50 repetições de um algoritmo e em seguida as 50 do outro, reaproveitando um único vetor alocado por tamanho. O bloco de configuração no início do `main.c` permite selecionar quais algoritmos, cenários e tamanhos executar, o que permitiu realizar a coleta em várias execuções, entre 21 e 24 de outubro de 2026.

Cada combinação gera um arquivo CSV próprio, no caminho `resultados/<algoritmo>/<cenário>/<n>.csv`, por exemplo `resultados/merge/medio/0100000.csv`.

Cada arquivo tem um cabeçalho e uma linha por repetição, com os campos `algoritmo`, `cenario`, `n`, `repeticao` e `tempo_s`.

2.6 Estatística descritiva

O processamento dos dados foi feito em Python, com as bibliotecas pandas (McKinney, 2010), NumPy (Harris *et al.*, 2020), SciPy (Virtanen *et al.*, 2020) e Matplotlib (Hunter, 2007), pelos programas `analise.py` e `docs/gerar_recursos.py` do projeto. Para cada combinação, com r repetições e tempos $t_1, \ldots, t_r$, foram calculados a média $\bar{t}$, o desvio padrão amostral s, os valores mínimo e máximo e o intervalo de confiança de 95 % para a média, dado pela Equação 2.1. A equação usa a distribuição t de Student, que, segundo Montgomery e Runger (2016), é a adequada quando a variância populacional é desconhecida:

$$\bar{t} \pm t_{0{,}975;\, r-1} \cdot \frac{s}{\sqrt{r}} \tag{2.1}$$

Com $r = 50$ repetições, o quantil é $t_{0{,}975;\, 49} \approx 2{,}0096$. O intervalo indica a faixa em que se espera encontrar o tempo médio verdadeiro da combinação, com 95 % de confiança.

Para decidir se a diferença entre os dois algoritmos é significativa em um dado n, foram usados dois critérios em conjunto: a ausência de sobreposição entre os intervalos de confiança das duas médias e o teste t de Welch, que compara duas médias sem supor variâncias iguais (Welch, 1947), com nível de significância $\alpha = 0{,}05$.

2.7 Regressão polinomial e indicadores de crescimento

Para cada combinação de algoritmo e cenário, foram ajustados polinômios de grau 1, 2 e 3 às médias de tempo em função de n (dez pontos por combinação), pelo método dos mínimos quadrados, com a função `numpy.polyfit` (Harris *et al.*, 2020). Para evitar problemas numéricos com potências muito grandes de n, o ajuste foi feito sobre a variável $x = n/10^6$, e os coeficientes foram depois convertidos para a variável n. A qualidade de cada ajuste foi medida pelo coeficiente de determinação R^2. Como a função teórica do Merge Sort não é polinomial, foi ajustado também, para comparação, o modelo $T(n) = a\, n \log_2 n + b$, pelo mesmo método.

Foram calculados ainda dois indicadores complementares:

a) **razão ao dobrar** n: $T(2n)/T(n)$ para os pares de tamanhos em que $2n$ também foi medido. Se $T(n) \propto n^k$, essa razão vale 2^k: 2 para crescimento linear e 4 para crescimento quadrático. Para $T(n) \propto n \log_2 n$, vale $2 \log_2(2n) / \log_2 n$, ou seja, entre 2,10 e 2,20 no intervalo estudado;

b) **expoente empírico** k: inclinação da reta ajustada por mínimos quadrados aos pontos $(\log n, \log T)$. Se $T(n) \propto n^k$, os pontos ficam sobre uma reta de inclinação k.

3 RESULTADOS

Este capítulo apresenta os dados coletados, sem interpretação. As tabelas 1 a 6 trazem as estatísticas de cada combinação de algoritmo, cenário e tamanho de entrada, com todos os tempos em milissegundos. A coluna *Rep.* indica o número de repetições válidas.

3.1 Estatísticas do Insertion Sort

Tabela 1 – Insertion Sort, melhor caso: estatísticas do tempo de execução (ms)

n	Média	Desvio padrão	IC 95 % (±)	Mínimo	Máximo	Rep.
1.000	0,0007371	0,00007556	0,00002147	0,0007100	0,001222	50
2.000	0,001844	0,0003830	0,0001088	0,001370	0,002513	50
5.000	0,005648	0,002054	0,0005839	0,003382	0,01606	50
10.000	0,01531	0,01081	0,003071	0,006737	0,06108	50
20.000	0,01799	0,004085	0,001161	0,01341	0,03736	50
50.000	0,04089	0,007348	0,002088	0,03352	0,05854	50
100.000	0,1161	0,03471	0,009866	0,06902	0,2265	50
200.000	0,1542	0,03028	0,008605	0,1338	0,2525	50
500.000	0,4660	0,1307	0,03715	0,3345	0,7889	50
1.000.000	0,8295	0,1872	0,05319	0,6718	1,316	50

Fonte: Elaborada pelo autor (2026).

Tabela 2 – Insertion Sort, caso médio: estatísticas do tempo de execução (ms)

n	Média	Desvio padrão	IC 95 % (±)	Mínimo	Máximo	Rep.
1.000	0,09701	0,01541	0,004380	0,07151	0,1399	50
2.000	0,3420	0,1189	0,03378	0,2686	0,9552	50
5.000	1,690	0,1051	0,02988	1,605	2,162	50
10.000	7,049	1,096	0,3116	6,377	11,01	50
20.000	27,23	1,667	0,4737	25,56	33,48	50
50.000	172,2	6,569	1,867	161,9	192,6	50
100.000	673,8	12,65	3,596	656,7	718,2	50
200.000	2.698	31,29	8,891	2.648	2.801	50
500.000	16.856	198,4	56,39	16.551	17.864	50
1.000.000	66.828	665,0	189,0	65.699	69.661	50

Fonte: Elaborada pelo autor (2026).

Tabela 3 – Insertion Sort, pior caso: estatísticas do tempo de execução (ms)

n	Média	Desvio padrão	IC 95 % (±)	Mínimo	Máximo	Rep.
1.000	0,1536	0,02128	0,006047	0,1344	0,2009	50
2.000	0,5419	0,04133	0,01175	0,5197	0,7560	50
5.000	3,561	0,4619	0,1313	3,191	5,203	50
10.000	13,70	0,9384	0,2667	12,82	18,24	50
20.000	56,60	3,111	0,8842	52,27	64,37	50
50.000	344,0	6,694	1,902	331,5	357,0	50
100.000	1.419	67,37	19,15	1.341	1.651	50
200.000	5.702	245,7	69,83	5.398	6.468	50
500.000	34.049	1.131	321,5	32.779	36.963	50
1.000.000	133.541	1.885	535,7	131.812	144.296	50

Fonte: Elaborada pelo autor (2026).

3.2 Estatísticas do Merge Sort

Tabela 4 – Merge Sort, melhor caso: estatísticas do tempo de execução (ms)

n	Média	Desvio padrão	IC 95 % (±)	Mínimo	Máximo	Rep.
1.000	0,02718	0,004353	0,001237	0,02197	0,03746	50
2.000	0,05381	0,006466	0,001838	0,04699	0,07788	50
5.000	0,1553	0,02699	0,007670	0,1200	0,2443	50
10.000	0,2592	0,01848	0,005252	0,2414	0,3109	50
20.000	0,5038	0,01620	0,004605	0,4875	0,5793	50
50.000	1,395	0,1898	0,05393	1,242	2,383	50
100.000	2,820	0,3414	0,09703	2,575	3,933	50
200.000	5,979	0,7643	0,2172	5,266	8,522	50
500.000	15,57	1,484	0,4217	13,80	19,67	50
1.000.000	31,92	3,530	1,003	27,96	49,70	50

Fonte: Elaborada pelo autor (2026).

Tabela 5 – Merge Sort, caso médio: estatísticas do tempo de execução (ms)

n	Média	Desvio padrão	IC 95 % (±)	Mínimo	Máximo	Rep.
1.000	0,07454	0,009176	0,002608	0,05847	0,1105	50
2.000	0,1291	0,005195	0,001476	0,1230	0,1460	50
5.000	0,3806	0,05492	0,01561	0,3317	0,5769	50
10.000	0,7936	0,2022	0,05745	0,7009	1,979	50
20.000	1,627	0,1331	0,03783	1,499	1,980	50
50.000	4,381	1,161	0,3299	4,043	12,37	50
100.000	9,222	0,9726	0,2764	8,534	14,49	50
200.000	19,86	2,046	0,5815	18,22	30,77	50
500.000	51,78	2,631	0,7478	48,46	60,76	50
1.000.000	105,7	2,482	0,7055	102,6	115,2	50

Fonte: Elaborada pelo autor (2026).

Tabela 6 – Merge Sort, pior caso: estatísticas do tempo de execução (ms)

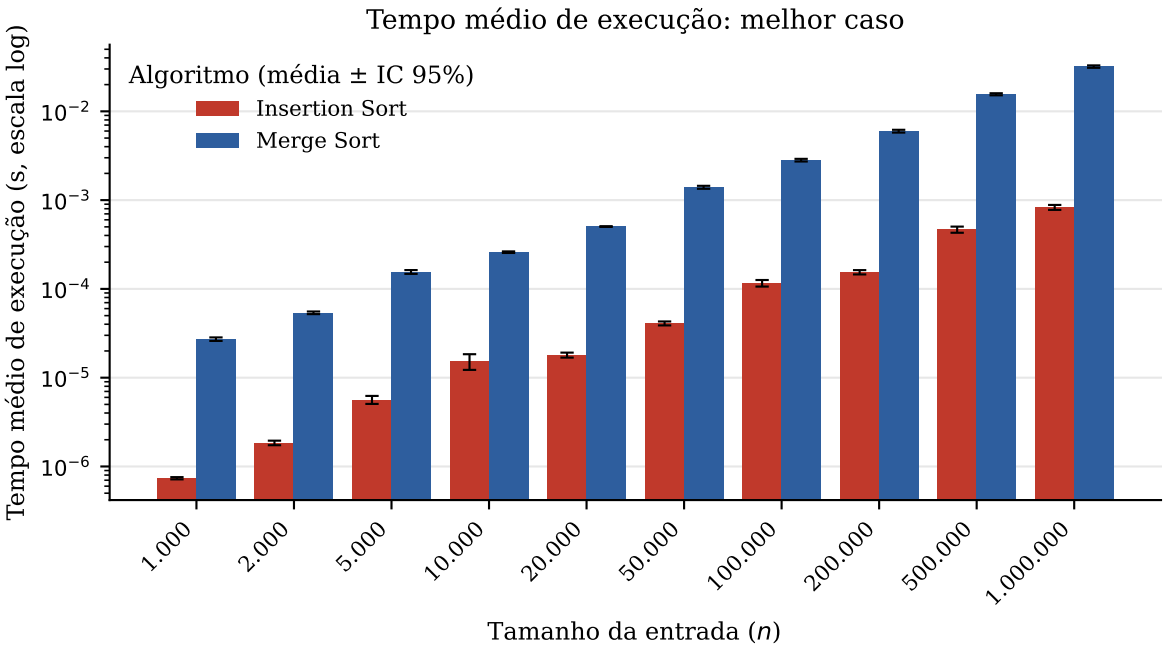
n	Média	Desvio padrão	IC 95 % (±)	Mínimo	Máximo	Rep.
1.000	0,02786	0,003802	0,001080	0,02229	0,03828	50
2.000	0,06500	0,01308	0,003718	0,05883	0,1460	50
5.000	0,1452	0,01518	0,004315	0,1320	0,2031	50
10.000	0,2721	0,04613	0,01311	0,2499	0,5296	50
20.000	0,5390	0,07460	0,02120	0,4755	0,7793	50
50.000	1,299	0,05001	0,01421	1,242	1,514	50
100.000	2,829	0,2738	0,07780	2,557	4,315	50
200.000	6,192	0,9091	0,2584	5,290	7,947	50
500.000	14,10	0,5685	0,1616	13,60	16,89	50
1.000.000	31,65	2,434	0,6916	29,21	41,12	50

Fonte: Elaborada pelo autor (2026).

3.3 Gráficos comparativos

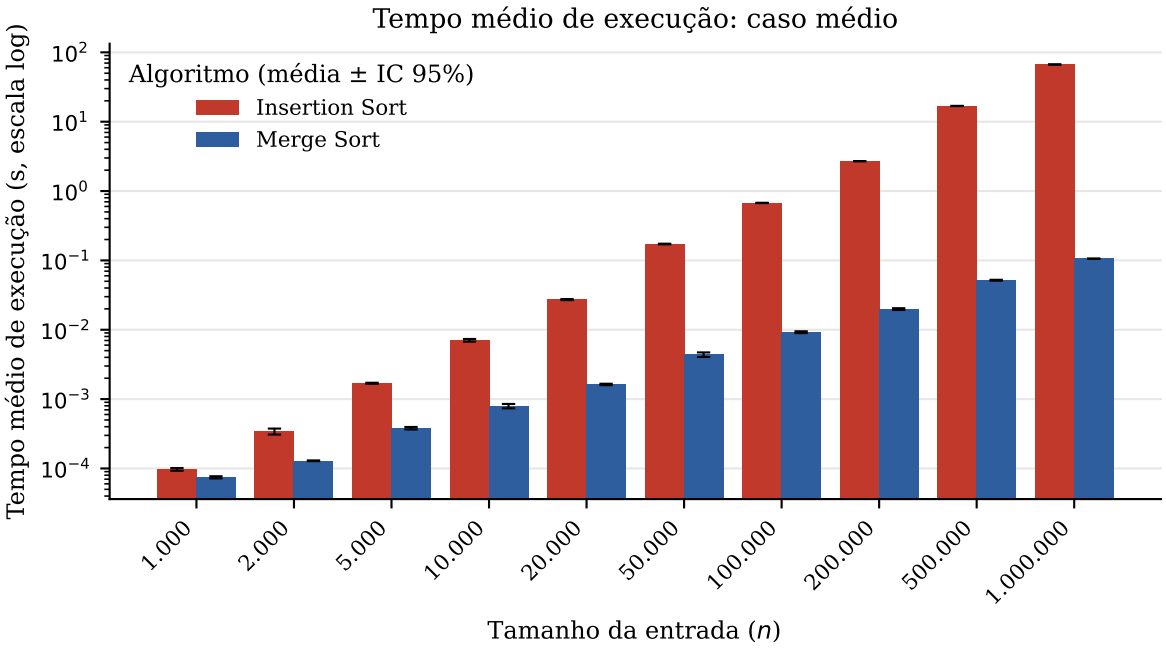
As figuras 1 a 3 mostram, para cada cenário, o tempo médio de execução de cada algoritmo em função de n, com barras de erro correspondentes ao intervalo de confiança de 95 %. Como os tempos variam ao longo de mais de seis ordens de grandeza, o eixo vertical está em escala logarítmica.

Figura 1 – Tempo médio de execução no melhor caso (vetor crescente)



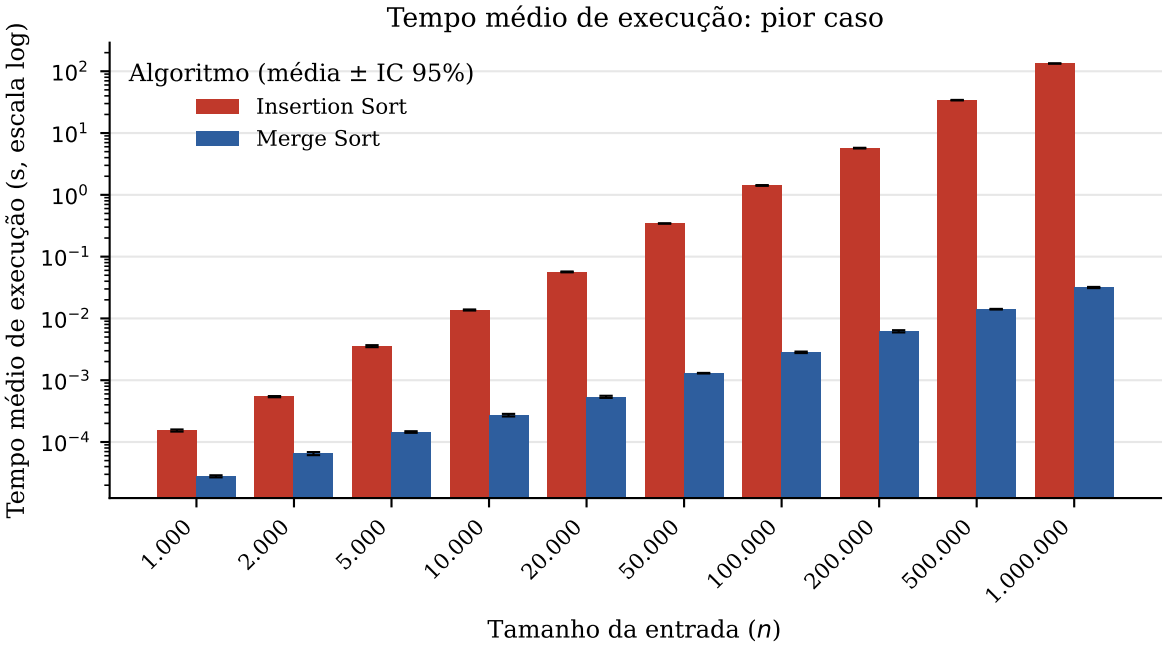
Fonte: Elaborada pelo autor (2026).

Figura 2 – Tempo médio de execução no caso médio (vetor aleatório)



Fonte: Elaborada pelo autor (2026).

Figura 3 – Tempo médio de execução no pior caso (vetor decrescente)



Fonte: Elaborada pelo autor (2026).

4 ANÁLISE

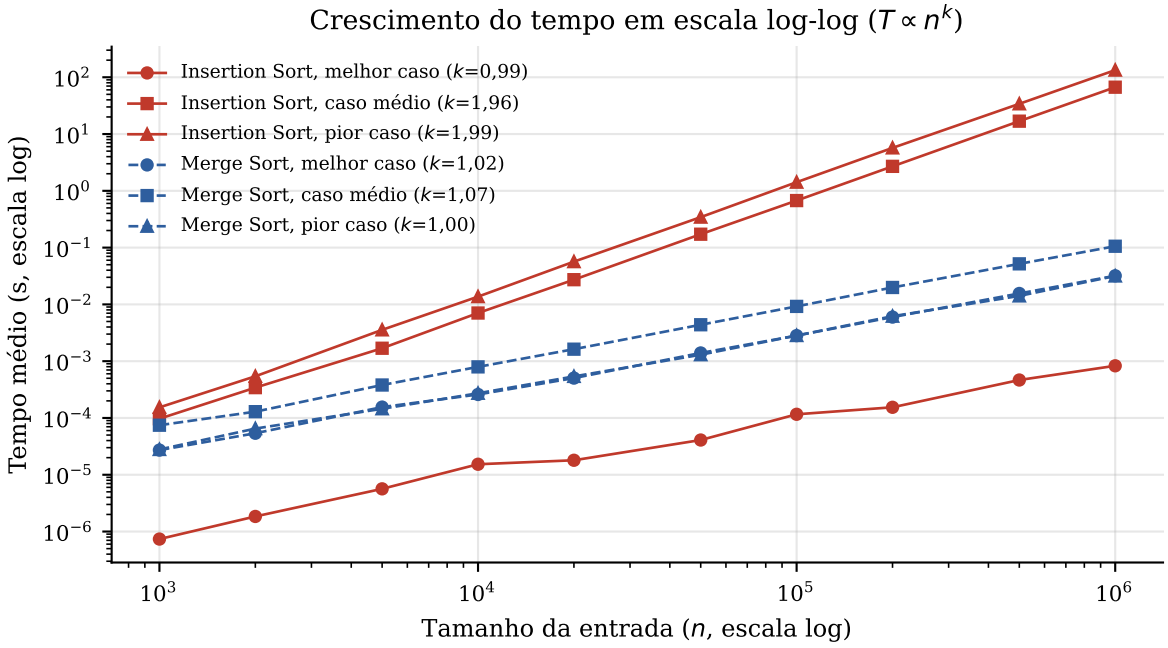
Este capítulo interpreta os resultados do Capítulo 3 à luz da análise teórica dos algoritmos, respondendo às questões propostas no enunciado.

4.1 Ordens de crescimento observadas

Nos gráficos comparativos, a diferença de comportamento entre os algoritmos é visível pela inclinação das barras. Como o eixo vertical é logarítmico, uma mesma razão de crescimento aparece como um mesmo degrau de altura. Nos casos médio e pior (figuras 2 e 3), as barras do Insertion Sort sobem muito mais rápido que as do Merge Sort. No caso médio, o tempo do Insertion Sort passa de 0,097 ms com $n = 1.000$ para 66.828 ms (cerca de 67 segundos) com $n = 1.000.000$, um aumento de aproximadamente $6{,}9 \times 10^5$ vezes para um aumento de 10^3 vezes em n, compatível com crescimento quadrático ($10^{3 \cdot 2} = 10^6$). No mesmo intervalo, o tempo do Merge Sort passa de 0,075 ms para 105,7 ms, um aumento de cerca de 1.400 vezes, pouco acima do aumento linear de 10^3 vezes, como esperado de $n \log n$.

A Figura 4 reúne as seis séries em escala log-log, na qual uma função $T(n) \propto n^k$ aparece como uma reta de inclinação k. A Tabela 7 apresenta essas inclinações, os expoentes empíricos.

Figura 4 – Tempo médio em função de n em escala log-log



Fonte: Elaborada pelo autor (2026).

Tabela 7 – Expoentes empíricos k, obtidos pela inclinação no gráfico log-log

Algoritmo	Cenário	Expoente k	Expoente esperado
Insertion Sort	Melhor caso	0,987	1 ($\Theta(n)$)
Insertion Sort	Caso médio	1,959	2 ($\Theta(n^2)$)
Insertion Sort	Pior caso	1,991	2 ($\Theta(n^2)$)
Merge Sort	Melhor caso	1,021	$\approx$ 1,10 ($\Theta(n \log n)$)
Merge Sort	Caso médio	1,066	$\approx$ 1,10 ($\Theta(n \log n)$)
Merge Sort	Pior caso	1,004	$\approx$ 1,10 ($\Theta(n \log n)$)

Fonte: Elaborada pelo autor (2026).

Os expoentes do Insertion Sort nos casos médio (1,959) e pior (1,991) são muito próximos de 2, e o do melhor caso (0,987) é muito próximo de 1, confirmando que o algoritmo é $\Theta(n^2)$ nos dois primeiros cenários e $\Theta(n)$ no último (Cormen *et al.*, 2012). A ligeira diferença em relação a 2 se explica pelos termos de ordem inferior, que pesam relativamente mais nos menores valores de n.

Para o Merge Sort, os expoentes ficaram entre 1,004 e 1,066, abaixo do valor de aproximadamente 1,10 que uma função exatamente proporcional a $n \log_2 n$ teria nesse intervalo. A diferença é pequena e indica que, nos menores tamanhos, custos que não crescem com $n \log n$, como as chamadas de função e as alocações de memória, pesam proporcionalmente mais e achatam a inclinação. Ainda assim, o crescimento é claramente

muito inferior ao quadrático e compatível com $\Theta(n \log n)$. Como o fator $\log_2 n$ varia apenas de cerca de 10 para cerca de 20 em todo o intervalo estudado, distinguir com precisão $n \log n$ de n exigiria tamanhos de entrada ainda maiores.

4.2 Comportamento ao dobrar o tamanho da entrada

A Tabela 8 mostra a razão $T(2n)/T(n)$ para os seis pares de tamanhos em que o dobro de n foi medido, ao lado da razão esperada pela teoria.

Tabela 8 – Razão $T(2n)/T(n)$ entre os tempos médios ao dobrar n

	Insertion Sort			Merge Sort		
$n \to 2n$	Melhor caso	Caso médio	Pior caso	Melhor caso	Caso médio	Pior caso
1.000 → 2.000	2,50	3,53	3,53	1,98	1,73	2,33
5.000 → 10.000	2,71	4,17	3,85	1,67	2,09	1,87
10.000 → 20.000	1,18	3,86	4,13	1,94	2,05	1,98
50.000 → 100.000	2,84	3,91	4,13	2,02	2,11	2,18
100.000 → 200.000	1,33	4,00	4,02	2,12	2,15	2,19
500.000 → 1.000.000	1,78	3,96	3,92	2,05	2,04	2,24
Esperada (teoria)	2,0	4,0	4,0	$\approx 2{,}1$	$\approx 2{,}1$	$\approx 2{,}1$

Fonte: Elaborada pelo autor (2026).

No Insertion Sort, nos casos médio e pior, as razões ficaram entre 3,53 e 4,17, oscilando em torno de 4, o valor esperado para um algoritmo quadrático: dobrar a entrada quadruplica o tempo, porque o número de deslocamentos é proporcional ao número de inversões do vetor, que é da ordem de n^2 (Knuth, 1998). Os valores mais afastados de 4 ocorreram nos menores tamanhos (1.000 → 2.000), em que os tempos são inferiores a 1 ms e mais sensíveis a ruído e a efeitos de cache. Do par 5.000 → 10.000 em diante, todas as razões ficaram entre 3,85 e 4,17.

No melhor caso, as razões do Insertion Sort variaram bastante (de 1,18 a 2,84), embora em torno de 2, o valor esperado para crescimento linear. Essa irregularidade não contradiz a teoria: nesse cenário os tempos são da ordem de microssegundos (menos de 1 ms mesmo com um milhão de elementos), de modo que pequenas perturbações do sistema têm grande efeito relativo, como mostra a Seção 4.5.

No Merge Sort, as razões ficaram entre 1,67 e 2,33 nos três cenários. A partir do par 50.000 → 100.000, todas ficaram entre 2,02 e 2,24, próximas do valor esperado, de aproximadamente 2,1. A razão ligeiramente superior a 2 é a assinatura do fator logarítmico: dobrar n dobra o trabalho de cada nível de intercalação e acrescenta um nível à recursão.

No maior par medido, 500.000 → 1.000.000, o comportamento se manteve nos três cenários. No pior caso, por exemplo, o tempo do Insertion Sort passou de 34,0 s para 133,5 s (razão de 3,92) e o do Merge Sort de 14,1 ms para 31,7 ms (razão de 2,24).

4.3 Comparação entre os algoritmos

As tabelas 9 a 11 comparam os tempos médios dos dois algoritmos em cada tamanho, com a razão entre eles (Insertion/Merge) e os resultados dos dois critérios de significância descritos na Seção 2.6.

Tabela 9 – Comparação entre os algoritmos no melhor caso

n	Insertion (ms)	Merge (ms)	Razão I/M	IC se sobrepõem?	Diferença significativa?
1.000	0,0007371	0,02718	0,0271	Não	Sim
2.000	0,001844	0,05381	0,0343	Não	Sim
5.000	0,005648	0,1553	0,0364	Não	Sim
10.000	0,01531	0,2592	0,0591	Não	Sim
20.000	0,01799	0,5038	0,0357	Não	Sim
50.000	0,04089	1,395	0,0293	Não	Sim
100.000	0,1161	2,820	0,0412	Não	Sim
200.000	0,1542	5,979	0,0258	Não	Sim
500.000	0,4660	15,57	0,0299	Não	Sim
1.000.000	0,8295	31,92	0,0260	Não	Sim

Fonte: Elaborada pelo autor (2026).

Tabela 10 – Comparação entre os algoritmos no caso médio

n	Insertion (ms)	Merge (ms)	Razão I/M	IC se sobrepõem?	Diferença significativa?
1.000	0,09701	0,07454	1,30	Não	Sim
2.000	0,3420	0,1291	2,65	Não	Sim
5.000	1,690	0,3806	4,44	Não	Sim
10.000	7,049	0,7936	8,88	Não	Sim
20.000	27,23	1,627	16,7	Não	Sim
50.000	172,2	4,381	39,3	Não	Sim
100.000	673,8	9,222	73,1	Não	Sim
200.000	2.698	19,86	136	Não	Sim
500.000	16.856	51,78	326	Não	Sim
1.000.000	66.828	105,7	632	Não	Sim

Fonte: Elaborada pelo autor (2026).

Tabela 11 – Comparação entre os algoritmos no pior caso

n	Insertion (ms)	Merge (ms)	Razão I/M	IC se sobrepõem?	Diferença significativa?
1.000	0,1536	0,02786	5,51	Não	Sim
2.000	0,5419	0,06500	8,34	Não	Sim
5.000	3,561	0,1452	24,5	Não	Sim
10.000	13,70	0,2721	50,3	Não	Sim
20.000	56,60	0,5390	105	Não	Sim
50.000	344,0	1,299	265	Não	Sim
100.000	1.419	2,829	502	Não	Sim
200.000	5.702	6,192	921	Não	Sim
500.000	34.049	14,10	2.414	Não	Sim
1.000.000	133.541	31,65	4.219	Não	Sim

Fonte: Elaborada pelo autor (2026).

Em todos os 30 pares comparados, os intervalos de confiança não se sobrepõem e o teste de Welch rejeitou a igualdade das médias ($p < 10^{-12}$ em todos os casos). Assim, a diferença entre os algoritmos foi estatisticamente significativa em todos os tamanhos testados, em todos os cenários. O que muda entre os cenários é qual algoritmo é o mais rápido:

a) **caso médio**: o Merge Sort já é mais rápido no menor tamanho testado, mas por pouco: com $n = 1.000$, o Insertion Sort leva 0,097 ms e o Merge Sort 0,075 ms (razão de 1,30). A vantagem do Merge Sort cresce rapidamente: a razão passa de 8,9 com $n = 10.000$ para 73 com $n = 100.000$ e 632 com $n = 1.000.000$. Como a razão entre as duas funções teóricas é $n^2/(n \log n) = n/\log n$, a vantagem do Merge Sort aumenta sem limite à medida que n cresce;

b) **pior caso**: o Merge Sort é mais rápido em todos os tamanhos, já 5,5 vezes com $n = 1.000$, chegando a cerca de 4.219 vezes com $n = 1.000.000$, em que o Insertion Sort leva 133,5 s e o Merge Sort 31,7 ms;

c) **melhor caso**: o Insertion Sort é mais rápido em todos os tamanhos, executando entre 17 e 39 vezes mais rápido que o Merge Sort (razão I/M entre 0,026 e 0,059). Com o vetor já ordenado, o Insertion Sort apenas percorre o vetor fazendo uma comparação por elemento, em tempo $\Theta(n)$, enquanto o Merge Sort executa todas as divisões, cópias e intercalações, em tempo $\Theta(n \log n)$, independentemente da ordem da entrada.

Portanto, nos tamanhos medidos, o Merge Sort foi mais rápido que o Insertion Sort a partir do menor tamanho testado ($n = 1.000$) nos casos médio e pior, e o ponto de cruzamento entre as duas curvas está abaixo desse valor. Isso é coerente com a teoria: para valores pequenos de n, o Insertion Sort tende a ser mais rápido, porque suas constantes são menores. Ele não faz chamadas recursivas, não aloca memória e percorre o vetor de

forma sequencial, o que favorece o uso da cache. Por isso, é considerado eficiente para ordenar poucos elementos (Cormen *et al.*, 2012). A estreita margem com $n = 1.000$ no caso médio (razão de 1,30) indica que o cruzamento deve ocorrer em tamanhos da ordem de algumas centenas de elementos nesta máquina. Determinar esse ponto com precisão exigiria medir tamanhos abaixo de 1.000, o que não fez parte do experimento.

4.4 Diferenças entre os cenários

O Insertion Sort é fortemente sensível à configuração da entrada. Com $n = 1.000.000$, ordenar o vetor aleatório (66,8 s) levou cerca de 80.000 vezes mais tempo que ordenar o vetor crescente (0,83 ms). Entre o pior caso e o caso médio, a razão entre os tempos se estabilizou em torno de 2 a partir de $n = 5.000$ (entre 1,94 e 2,11). Esse valor tem explicação teórica direta: o número de deslocamentos do Insertion Sort é igual ao número de inversões do vetor, que é $n(n-1)/2$ no vetor decrescente e, em média, $n(n-1)/4$ em um vetor aleatório (Knuth, 1998), uma razão de exatamente 2.

O Merge Sort, por outro lado, tem o mesmo número de níveis de recursão e de cópias em qualquer cenário, como prevê a análise $\Theta(n \log n)$ válida para todos os casos. De fato, os tempos do melhor caso e do pior caso foram praticamente iguais (razão entre 0,83 e 1,10). O caso médio, porém, foi de 3,0 a 3,7 vezes mais lento que os outros dois a partir de $n = 20.000$, cerca de 3,3 vezes na maioria dos tamanhos. A diferença não altera a ordem de crescimento, mas mostra que as constantes dependem da entrada. Uma explicação plausível é que, com vetores já ordenados ou em ordem inversa, a intercalação esgota uma das metades rapidamente e realiza cerca de metade das comparações. Além disso, os resultados dessas comparações seguem um padrão regular, que o processador consegue prever. No vetor aleatório, o número de comparações é maior e seus resultados são imprevisíveis. Este experimento, no entanto, não mediu contagens de comparações nem eventos do processador para confirmar essa hipótese.

4.5 Variabilidade e intervalos de confiança

Houve variação entre as 50 execuções de uma mesma combinação, e ela foi maior, em termos relativos, nos tempos mais curtos. Medida pelo coeficiente de variação (desvio padrão dividido pela média), a variação foi de cerca de 1 % nas combinações mais longas, como o Insertion Sort no caso médio com $n = 1.000.000$ (desvio de 665 ms em uma média de 66.828 ms). Já nas combinações de poucos microssegundos ela foi bem maior: chegou a 71 % no Insertion Sort no melhor caso com $n = 10.000$, em que o tempo máximo foi nove vezes o mínimo. Em algumas combinações há valores máximos bem acima da média, como no Merge Sort no caso médio com $n = 50.000$ (máximo de 12,4 ms para média de 4,4 ms), o que caracteriza execuções pontualmente perturbadas.

Essa variação tem várias causas possíveis no ambiente utilizado: interrupções e trocas de contexto feitas pelo sistema operacional, que entram no tempo medido por `CLOCK_MONOTONIC`; o estado da cache e da memória no início de cada execução; o ajuste dinâmico da frequência do processador; outros programas em execução no computador; e a camada de virtualização do WSL 2. Essas perturbações têm duração aproximadamente fixa, por isso pesam muito mais em uma medição de microssegundos do que em uma de segundos. Esse comportamento é esperado em medições de tempo decorrido, e é por isso que a metodologia experimental recomenda repetir as medições e relatar sua dispersão (Johnson, 2002; McGeoch, 2012).

O intervalo de confiança de 95 % resume essa variabilidade do ponto de vista da média: se o experimento fosse repetido muitas vezes nas mesmas condições, cerca de 95 % dos intervalos calculados dessa forma conteriam o tempo médio verdadeiro. Os intervalos foram estreitos em relação às médias. No Insertion Sort no caso médio com $n = 1.000.000$, por exemplo, o intervalo é de 66.828 ± 189 ms, ou seja, $\pm 0{,}3$ %. Por isso, a média de 50 repetições foi uma estimativa precisa, mesmo com a dispersão das execuções individuais, e as diferenças entre algoritmos e entre tamanhos são muito maiores que a incerteza das médias. Os intervalos não indicam a faixa em que uma execução individual vai cair; essa faixa é descrita pelo desvio padrão e pelos valores mínimo e máximo.

Executar o experimento em outra máquina mudaria os tempos absolutos, porque eles dependem da velocidade do processador, da memória, do compilador e do sistema operacional. Essas diferenças, porém, afetam apenas as constantes multiplicativas: o número de operações que cada algoritmo executa em função de n continua o mesmo. Por isso, as razões ao dobrar n, os expoentes empíricos e as ordens de crescimento devem se manter. É justamente essa independência de máquina que a análise assintótica descreve.

4.6 Regressão polinomial

As tabelas 12 e 13 apresentam as funções obtidas pelos ajustes por mínimos quadrados descritos na Seção 2.7, com $T(n)$ em segundos. As figuras 5 a 7 mostram as médias observadas e as curvas ajustadas.

Tabela 12 – Funções obtidas pela regressão polinomial para o Insertion Sort

Cenário	Modelo	Função obtida, $T(n)$ em segundos	R^2
Melhor caso	Grau 1	$8{,}413 \times 10^{-10}\, n + 5{,}975 \times 10^{-6}$	0,995381
Melhor caso	Grau 2	$-1{,}390 \times 10^{-16}\, n^2 + 9{,}717 \times 10^{-10}\, n - 5{,}341 \times 10^{-7}$	0,997136
Melhor caso	Grau 3	$-4{,}858 \times 10^{-22}\, n^3 + 5{,}460 \times 10^{-16}\, n^2 + 7{,}653 \times 10^{-10}\, n + 4{,}278 \times 10^{-6}$	0,997862
Caso médio	Grau 1	$6{,}275 \times 10^{-5}\, n - 3{,}121$	0,932618
Caso médio	Grau 2	$6{,}636 \times 10^{-11}\, n^2 + 4{,}822 \times 10^{-7}\, n - 1{,}216 \times 10^{-2}$	0,999999
Caso médio	Grau 3	$-1{,}400 \times 10^{-18}\, n^3 + 6{,}834 \times 10^{-11}\, n^2 - 1{,}128 \times 10^{-7}\, n + 1{,}708 \times 10^{-3}$	1,000000
Pior caso	Grau 1	$1{,}255 \times 10^{-4}\, n - 6{,}183$	0,934231
Pior caso	Grau 2	$1{,}310 \times 10^{-10}\, n^2 + 2{,}571 \times 10^{-6}\, n - 4{,}566 \times 10^{-2}$	0,999999
Pior caso	Grau 3	$-2{,}531 \times 10^{-18}\, n^3 + 1{,}346 \times 10^{-10}\, n^2 + 1{,}495 \times 10^{-6}\, n - 2{,}059 \times 10^{-2}$	0,999999

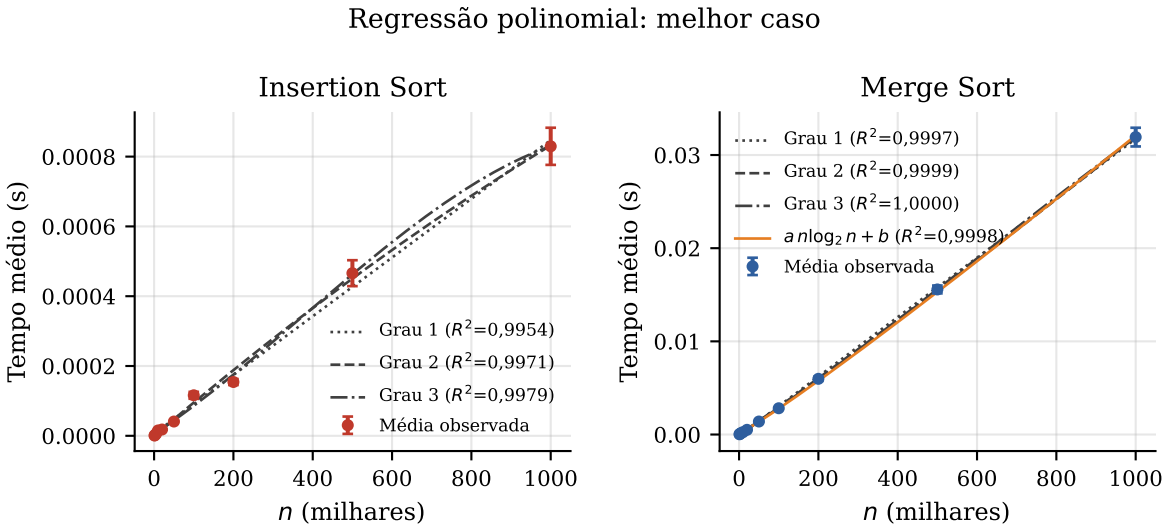
Fonte: Elaborada pelo autor (2026).

Tabela 13 – Funções obtidas pela regressão polinomial para o Merge Sort

Cenário	Modelo	Função obtida, $T(n)$ em segundos	R^2
Melhor caso	Grau 1	$3{,}189 \times 10^{-8}\, n - 1{,}528 \times 10^{-4}$	0,999723
Melhor caso	Grau 2	$1{,}877 \times 10^{-15}\, n^2 + 3{,}013 \times 10^{-8}\, n - 6{,}493 \times 10^{-5}$	0,999946
Melhor caso	Grau 3	$-4{,}407 \times 10^{-21}\, n^3 + 8{,}091 \times 10^{-15}\, n^2 + 2{,}826 \times 10^{-8}\, n - 2{,}128 \times 10^{-5}$	0,999988
Melhor caso	$n \log_2 n$	$1{,}604 \times 10^{-9}\, n \log_2 n + 1{,}130 \times 10^{-4}$	0,999811
Caso médio	Grau 1	$1{,}058 \times 10^{-7}\, n - 5{,}772 \times 10^{-4}$	0,999754
Caso médio	Grau 2	$5{,}350 \times 10^{-15}\, n^2 + 1{,}008 \times 10^{-7}\, n - 3{,}266 \times 10^{-4}$	0,999919
Caso médio	Grau 3	$-1{,}744 \times 10^{-20}\, n^3 + 2{,}994 \times 10^{-14}\, n^2 + 9{,}336 \times 10^{-8}\, n - 1{,}538 \times 10^{-4}$	0,999979
Caso médio	$n \log_2 n$	$5{,}322 \times 10^{-9}\, n \log_2 n + 3{,}056 \times 10^{-4}$	0,999751
Pior caso	Grau 1	$3{,}119 \times 10^{-8}\, n - 1{,}753 \times 10^{-4}$	0,997519
Pior caso	Grau 2	$5{,}192 \times 10^{-15}\, n^2 + 2{,}631 \times 10^{-8}\, n + 6{,}787 \times 10^{-5}$	0,999305
Pior caso	Grau 3	$1{,}497 \times 10^{-20}\, n^3 - 1{,}592 \times 10^{-14}\, n^2 + 3{,}268 \times 10^{-8}\, n - 8{,}046 \times 10^{-5}$	0,999809
Pior caso	$n \log_2 n$	$1{,}570 \times 10^{-9}\, n \log_2 n + 8{,}140 \times 10^{-5}$	0,998757

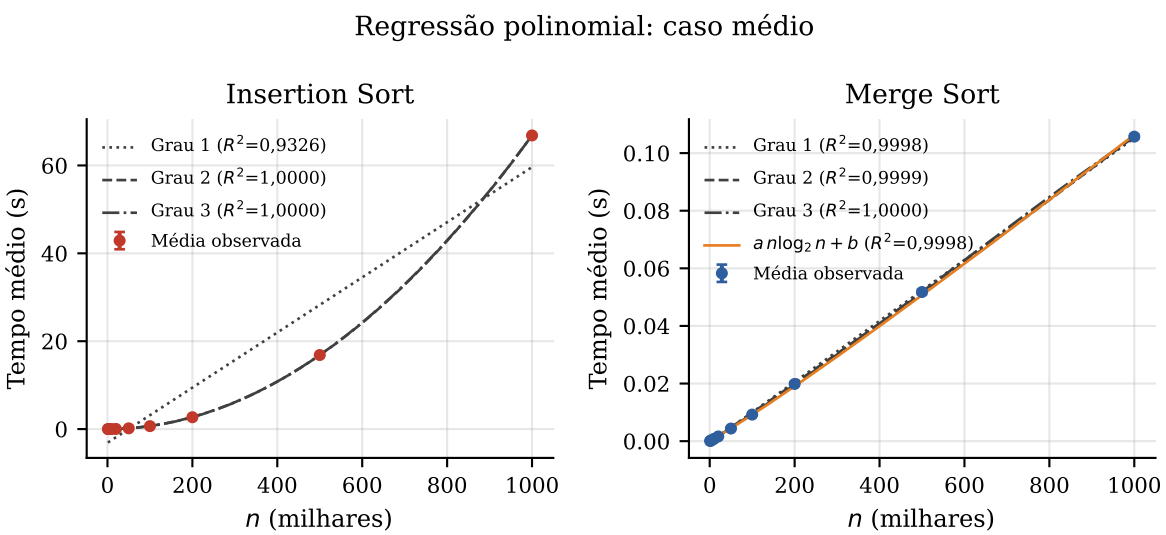
Fonte: Elaborada pelo autor (2026).

Figura 5 – Médias observadas e curvas ajustadas no melhor caso



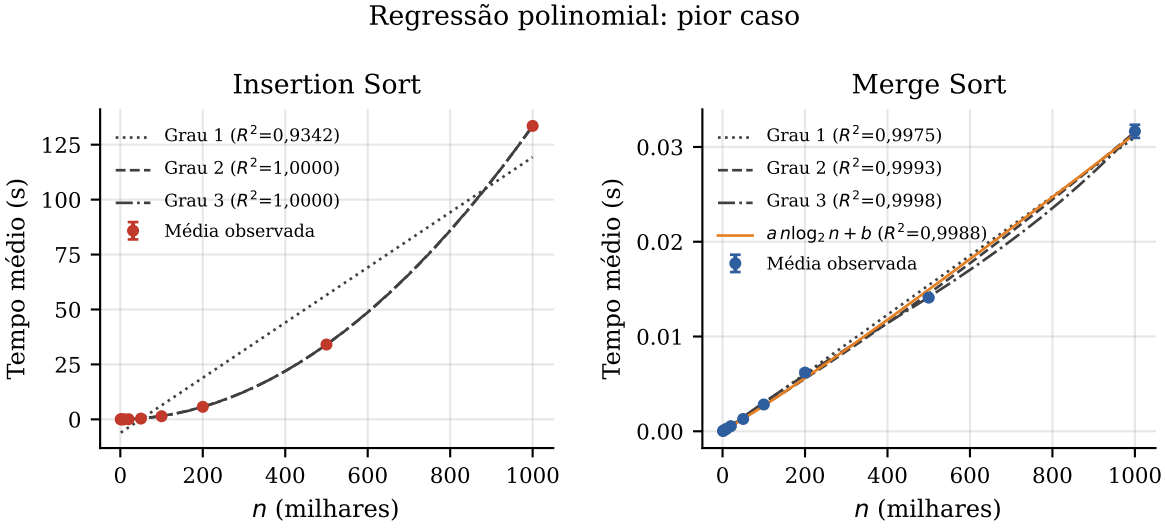
Fonte: Elaborada pelo autor (2026).

Figura 6 – Médias observadas e curvas ajustadas no caso médio



Fonte: Elaborada pelo autor (2026).

Figura 7 – Médias observadas e curvas ajustadas no pior caso



Fonte: Elaborada pelo autor (2026).

4.6.1 Insertion Sort

Nos casos médio e pior, o polinômio de grau 1 teve ajuste ruim (R^2 de 0,933 e 0,934), enquanto o de grau 2 se ajustou quase perfeitamente (R^2 de 0,999999 nos dois casos). As funções obtidas foram:

$$T_{\text{médio}}(n) \approx 6{,}636 \times 10^{-11}\, n^2 + 4{,}822 \times 10^{-7}\, n - 1{,}216 \times 10^{-2}$$
$$T_{\text{pior}}(n) \approx 1{,}310 \times 10^{-10}\, n^2 + 2{,}571 \times 10^{-6}\, n - 4{,}566 \times 10^{-2}$$

Essas funções têm a forma quadrática $T(n) = an^2 + bn + c$ prevista pela teoria para o Insertion Sort (Cormen *et al.*, 2012), e o termo quadrático domina: com $n = 1.000.000$, ele responde por mais de 98 % do valor da função nos dois cenários. Os coeficientes também têm interpretação física. O coeficiente quadrático do pior caso ($1{,}310 \times 10^{-10}$) é 1,97 vezes o do caso médio ($6{,}636 \times 10^{-11}$), muito próximo da razão de 2 entre $n^2/2$ e $n^2/4$ deslocamentos esperada pela contagem de inversões (Knuth, 1998). Dividindo o tempo do caso médio com $n = 1.000.000$ pelo número esperado de deslocamentos ($n^2/4$), obtém-se cerca de 0,27 ns por deslocamento, o custo de uma iteração do laço interno nesta máquina. Os coeficientes linear e constante são pequenos, e os termos constantes negativos não têm significado físico: são apenas artefatos do ajuste, irrelevantes diante do termo quadrático.

No melhor caso, o polinômio de grau 1, $T(n) \approx 8{,}413 \times 10^{-10}\, n + 5{,}975 \times 10^{-6}$, já se ajustou bem ($R^2 = 0{,}9954$). Isso é compatível com o comportamento linear previsto, a cerca de 0,8 ns por elemento. Os graus 2 e 3 aumentaram o R^2 apenas marginalmente (para 0,9971 e 0,9979), e o termo de maior grau ficou negativo nos dois casos, o que não

tem significado físico para um tempo de execução: é típico de um ajuste que passa a acompanhar o ruído das medições, e não uma tendência real.

O polinômio de grau 3 tem R^2 igual ou maior que o de grau 2 em todos os casos, por exemplo 1,000000 contra 0,999999 no caso médio. Isso, porém, não significa que o algoritmo tenha ordem de crescimento cúbica. Polinômios de grau maior contêm os de grau menor como caso particular: basta zerar o coeficiente adicional. Por isso, o ajuste por mínimos quadrados nunca piora quando se aumenta o grau, porque o termo extra sempre pode absorver parte do ruído. Com apenas dez pontos, um polinômio de grau suficientemente alto passaria exatamente por todos eles. O critério para escolher o modelo não deve ser apenas o maior R^2, e sim o menor grau que explica os dados com coeficientes coerentes. No Insertion Sort, o coeficiente cúbico obtido é desprezível e de sinal negativo ($-1{,}400 \times 10^{-18}$ no caso médio), e o coeficiente quadrático praticamente não muda em relação ao ajuste de grau 2 ($6{,}834 \times 10^{-11}$ contra $6{,}636 \times 10^{-11}$). Isso mostra que o termo cúbico não representa uma tendência real dos dados. Portanto, o grau que melhor descreve o Insertion Sort é 2 nos casos médio e pior, e 1 no melhor caso, em concordância com a teoria.

4.6.2 Merge Sort

Para o Merge Sort, todos os modelos obtiveram R^2 acima de 0,997, inclusive o de grau 1. No caso médio, o polinômio de grau 1 ($R^2 = 0{,}99975$) e o modelo $n \log_2 n$ ($R^2 = 0{,}99975$) tiveram ajustes praticamente idênticos:

$$T_{\text{médio}}(n) \approx 1{,}058 \times 10^{-7}\, n - 5{,}772 \times 10^{-4} \qquad \text{(grau 1)}$$
$$T_{\text{médio}}(n) \approx 5{,}322 \times 10^{-9}\, n \log_2 n + 3{,}056 \times 10^{-4} \qquad (n \log_2 n)$$

O polinômio de grau 2 melhorou um pouco o ajuste ($R^2 = 0{,}99992$), com coeficiente quadrático positivo e muito pequeno ($5{,}350 \times 10^{-15}$). Esse termo não indica crescimento quadrático: ele aproxima a leve curvatura para cima que o fator $\log n$ produz em relação a uma reta. Com $n = 1.000.000$, o termo quadrático corresponde a cerca de 5 % do valor da função, enquanto no Insertion Sort o termo quadrático corresponde a quase todo o tempo.

A regressão polinomial é, portanto, consistente com o modelo teórico linearítmico. No entanto, nesse intervalo de tamanhos ela não consegue distinguir com clareza $n \log n$ de um crescimento linear. Como o fator $\log_2 n$ apenas dobra (de aproximadamente 10 para 20) enquanto n é multiplicado por mil, as duas curvas são muito parecidas, e o ajuste por mínimos quadrados sobre as médias é dominado pelos maiores valores de n. No melhor e no pior caso, o modelo $n \log_2 n$ teve ajuste um pouco melhor que o linear (R^2 de 0,99981 contra 0,99972 e de 0,99876 contra 0,99752, respectivamente), mas a diferença é pequena e

está na mesma ordem da dispersão das medições: no pior caso com $n = 1.000.000$, por exemplo, o coeficiente de variação do Merge Sort foi de 7,7 %. As evidências mais claras do fator logarítmico são as razões ao dobrar n ligeiramente acima de 2 (Seção 4.2) e a grande diferença em relação ao comportamento quadrático do Insertion Sort, e não o R^2.

5 CONCLUSÃO

O experimento confirmou empiricamente as ordens de crescimento previstas pela análise teórica. O Insertion Sort apresentou crescimento quadrático nos casos médio e pior, com expoentes empíricos de 1,96 e 1,99, razão próxima de 4 ao dobrar n e polinômio de grau 2 com R^2 de 0,999999, e crescimento linear no melhor caso. O Merge Sort apresentou crescimento quase linear nos três cenários, com razão próxima de 2,1 ao dobrar n, compatível com $\Theta(n \log n)$. A relação próxima de 2 (1,97) entre os coeficientes quadráticos do pior caso e do caso médio do Insertion Sort coincidiu com a razão entre os números de inversões dos vetores decrescente e aleatório, o que mostra que a regressão capturou não apenas a forma da função, mas também a contagem de operações que a teoria descreve.

Na comparação entre os algoritmos, o Merge Sort foi significativamente mais rápido em todos os tamanhos testados nos casos médio e pior, com uma vantagem que cresceu de 1,3 vez com $n = 1.000$ para mais de 600 vezes com $n = 1.000.000$ no caso médio e para mais de 4.000 vezes no pior caso. O Insertion Sort, por sua vez, foi de 17 a 39 vezes mais rápido no melhor caso. Esses resultados mostram que a escolha do algoritmo depende do tamanho e da configuração da entrada, e não apenas da ordem de crescimento.

O experimento também evidenciou o que a análise assintótica deixa de fora: as constantes multiplicativas, que tornam o Insertion Sort competitivo em entradas pequenas; a influência da configuração da entrada nas constantes do Merge Sort, cujo caso médio foi cerca de 3 vezes mais lento que os demais apesar da mesma ordem de crescimento; e a variabilidade das medições, que exigiu repetições e intervalos de confiança para que as conclusões fossem confiáveis. Também mostrou um limite do método: no intervalo estudado, o R^2 da regressão não foi suficiente para distinguir $n \log n$ de crescimento linear, e indicadores como a razão ao dobrar n e o expoente empírico foram mais informativos. A análise teórica prevê como o custo cresce, e a análise empírica mostra quanto esse custo vale na prática e a partir de que ponto as previsões se tornam visíveis. As duas são complementares.

Como trabalhos futuros, sugere-se medir tamanhos abaixo de 1.000 para localizar o ponto de cruzamento entre os algoritmos no caso médio e contar comparações e movimentações, além do tempo, para verificar as explicações propostas para as diferenças entre cenários do Merge Sort.

REFERÊNCIAS

CORMEN, T. H. *et al. Algoritmos*: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

FREE SOFTWARE FOUNDATION. *Options that control optimization*: using the GNU Compiler Collection (GCC). 2026. Disponível em: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html. Acesso em: 22 out. 2026.

HARRIS, C. R. *et al.* Array programming with NumPy. *Nature*, v. 585, p. 357–362, 2020.

HUNTER, J. D. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90–95, 2007.

JOHNSON, D. S. A theoretician's guide to the experimental analysis of algorithms. In: GOLDWASSER, M. H.; JOHNSON, D. S.; MCGEOCH, C. C. (ed.). *Data structures, near neighbor searches, and methodology: fifth and sixth DIMACS implementation challenges*. Providence: American Mathematical Society, 2002. p. 215–250.

KNUTH, D. E. *The art of computer programming*. 2. ed. Reading: Addison-Wesley, 1998. v. 3. Sorting and searching.

LINUX MAN-PAGES PROJECT. *clock_getres(2)*: Linux manual page. 2026. Disponível em: https://man7.org/linux/man-pages/man2/clock_gettime.2.html. Acesso em: 22 out. 2026.

MCGEOCH, C. C. *A guide to experimental algorithmics*. Cambridge: Cambridge University Press, 2012.

MCKINNEY, W. Data structures for statistical computing in Python. In: *Proceedings of the 9th Python in Science Conference*. Austin: [s. n.], 2010. p. 56–61.

MONTGOMERY, D. C.; RUNGER, G. C. *Estatística aplicada e probabilidade para engenheiros*. 6. ed. Rio de Janeiro: LTC, 2016.

VIRTANEN, P. *et al.* SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, v. 17, p. 261–272, 2020.

WELCH, B. L. The generalization of "Student's" problem when several different population variances are involved. *Biometrika*, v. 34, n. 1/2, p. 28–35, 1947.

Apêndices

APÊNDICE A – REPRODUÇÃO DO EXPERIMENTO

Os comandos a seguir, executados na raiz do projeto, reproduzem o ciclo completo de compilação, execução, análise e geração deste relatório.

```
# 1. Compilar e executar o benchmark (grava os CSVs em resultados/)
make
./main

# 2. Preparar o ambiente Python e gerar tabelas e figuras
python3 -m venv venv
venv/bin/pip install -r requirements.txt
venv/bin/python analise.py                # saídas em saida/
venv/bin/python docs/gerar_recursos.py    # saídas em docs/recursos/

# 3. Compilar o relatório
cd docs && latexmk -pdf relatorio.tex
```

Os algoritmos, cenários e tamanhos executados pelo benchmark são definidos no bloco de configuração no início do arquivo `main.c`, pelos vetores `algoritmos_a_rodar`, `cenarios_a_rodar` e `tamanhos_a_rodar`. Cada execução sobrescreve apenas os arquivos das combinações selecionadas.